

E17 — Initialization Type Comparison

Overview

This experiment investigates how different **initialization strategies** affect the convergence behavior of **Muon** (spectral normalization optimizer) versus **SGD** (standard gradient descent with momentum).

Three initialization types are compared:

1. **Random**: Standard Gaussian initialization ($X^{(0)} = 0.01 \cdot Z$ where $Z_{ij} \sim \mathcal{N}(0, 1)$)
2. **Orthogonal**: Orthogonal matrix via QR decomposition, scaled by 0.01 — provides a well-conditioned starting point with unit-norm columns
3. **Spectral**: Product of two orthogonal matrices ($UV^\top$), scaled by 0.01 — provides a structured low-rank-like starting point

Why this matters: The choice of initialization can significantly impact which basin of attraction the optimization lands in, especially in non-convex landscapes. Muon's spectral normalization may interact differently with structured initializations (orthogonal, spectral) compared to random initialization, since the spectral structure of the initial matrix affects the early gradient directions.

Experiment ID: E17 | **Total runs:** 48 (2 algorithms *imes* 3 init types *imes* 8 seeds)

Scientific Question

Hypothesis

- **Null Hypothesis (H_0):** Initialization type has no differential effect on Muon vs SGD — both algorithms benefit (or suffer) equally from orthogonal and spectral initialization.
- **Alternative Hypothesis (H_1):** Muon's spectral normalization mechanism interacts differently with structured initializations than SGD's gradient descent, leading to measurably different performance improvements from orthogonal/spectral initialization.

Specific Questions

1. Does **orthogonal initialization** help either algorithm more than random?
2. Does **spectral initialization** (structured low-rank start) benefit Muon disproportionately?

- 3. Which algorithm shows greater consistency (lower variance) across initialization types?
- 4. Is there a statistically significant difference between any pair of initialization types for either algorithm?

Key Metrics

- K_ϵ : Iterations to reach loss threshold $\epsilon = 0.01$
- min loss: Best loss achieved during optimization
- I_{conv} : Convergence flag
- time_s : Wall-clock time in seconds

Experimental Design

Parameter	Value
Problem	Matrix Sensing (MS)
Matrix dimension d	50
Target rank r	5
Learning rate η	0.01
Measurement samples m	$2dr = 500$
Measurement distribution	Normal (Gaussian)
Noise	0 (noiseless)
Iteration budget	2000
Convergence threshold ϵ	0.01
Initialization types	{random, orthogonal, spectral}
Seeds per configuration	8
Algorithms	Muon-Exact, SGD

Initialization Construction

Type	Formula	Properties
random	$X^{(0)} = 0.01 \cdot Z, Z_{ij} \sim \mathcal{N}(0, 1)$	Unstructured, isotropic
orthogonal	$X^{(0)} = 0.01 \cdot Q, Q$ from QR of random matrix	Well-conditioned, $Q^\top Q = I$

Type	Formula	Properties
spectral	$X^{(0)} = 0.01 \cdot UV^\top, U, V$ from QR	Low-rank structure, $\ UV^\top\ _2 = 1$

```
In [ ]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
import warnings
warnings.filterwarnings('ignore')

# Color scheme
MUON_COLOR = '#2E86AB' # Blue
SGD_COLOR = '#F1F01' # Orange

# Set plotting style
sns.set_style("whitegrid")
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['font.size'] = 11

print("Libraries imported successfully.")
```

```
In [ ]: # Load E17 data
df = pd.read_csv('../results_v3/E17_detailed_results.csv')

print(f"Shape: {df.shape}")
print(f"Columns: {list(df.columns)}")
print(f"\nAlgorithms: {df['algo'].unique()}")
print(f"Init types: {df['init_type'].unique()}")
print(f"Seeds: {sorted(df['seed'].unique())}")
print("\nFirst few rows:")
df.head()
```

```
In [ ]: # Data quality check
print("=== Missing values ===")
print(df.isnull().sum())
print(f"\n=== Summary by algorithm and init_type ===")
summary = df.groupby(['algo', 'init_type'])['K_epsilon'].agg(['count', 'mean'])
print(summary)
print(f"\n=== Convergence status ===")
print(df.groupby(['algo', 'init_type'])['I_conv'].mean())
```

Exploratory Data Analysis

We examine the distribution of convergence iterations K_ϵ across initialization types for both algorithms. This helps us identify which initialization strategy leads to the fastest convergence for each algorithm.

```
In [ ]: # Detailed summary table
print("=" * 85)
print(f"{'Algorithm':<14} {'Init Type':>12} {'N':>4} {'Mean K_eps':>12} {'St
print("=" * 85)

for algo in ['Muon-Exact', 'SGD']:
    for it in df['init_type'].unique():
        sub = df[(df['algo'] == algo) & (df['init_type'] == it)]
        print(f"{'algo':<14} {'it':>12} {'len(sub)':>4} {sub['K_epsilon'].mean():>
              f"{'sub['K_epsilon'].std():>12.1f} {sub['I_conv'].mean():>10.1%

# Best init type per algorithm
print("\n=== Best initialization type per algorithm (lowest mean K_epsilon)
best = df.groupby(['algo', 'init_type'])['K_epsilon'].mean().reset_index()
for algo in ['Muon-Exact', 'SGD']:
    sub = best[best['algo'] == algo]
    best_row = sub.loc[sub['K_epsilon'].idxmin()]
    print(f"{'algo':<14} {'best_row['init_type']':>12} (K_eps={best_row['K_epsilon']:.1
```

Comparative Analysis: Muon vs SGD

We compare the two algorithms head-to-head for each initialization type, computing paired t-tests and efficiency ratios.

```
In [ ]: # Head-to-head comparison by init_type
print("=" * 90)
print(f"{'Init Type':>12} {'Muon K_eps':>12} {'SGD K_eps':>12} {'Ratio':>8}
print("=" * 90)

for it in df['init_type'].unique():
    muon_k = df[(df['algo'] == 'Muon-Exact') & (df['init_type'] == it)]['K_e
    sgd_k = df[(df['algo'] == 'SGD') & (df['init_type'] == it)]['K_epsilon']

    ratio = muon_k.mean() / sgd_k.mean()
    t_stat, p_val = stats.ttest_rel(muon_k, sgd_k)
    sig = "****" if p_val < 0.001 else "***" if p_val < 0.01 else "*" if p_val

    print(f"{'it':>12} {muon_k.mean():>12.1f} {sgd_k.mean():>12.1f} {ratio:>8.

print("\n=== ANOVA: Effect of init_type within each algorithm ===")
for algo in ['Muon-Exact', 'SGD']:
    sub = df[df['algo'] == algo]
    groups = [sub[sub['init_type'] == it]['K_epsilon'].values for it in sub[
    f_stat, p_val = stats.f_oneway(*groups)
    print(f"{'algo':<14} F={f_stat:.3f}, p={p_val:.6f} ({'significant' if p_val <
```

Visualization 1: K_ϵ by Initialization Type (Grouped Bar)

This plot compares the mean convergence iterations across initialization types for both algorithms, with error bars showing ± 1 standard deviation.

```
In [ ]: fig, ax = plt.subplots(figsize=(10, 6))

init_types = df['init_type'].unique()
x = np.arange(len(init_types))
width = 0.35

muon_means = [df[(df['algo'] == 'Muon-Exact') & (df['init_type'] == it)]['K_epsilon'].mean() for it in init_types]
muon_stds = [df[(df['algo'] == 'Muon-Exact') & (df['init_type'] == it)]['K_epsilon'].std() for it in init_types]
sgd_means = [df[(df['algo'] == 'SGD') & (df['init_type'] == it)]['K_epsilon'].mean() for it in init_types]
sgd_stds = [df[(df['algo'] == 'SGD') & (df['init_type'] == it)]['K_epsilon'].std() for it in init_types]

bars1 = ax.bar(x - width/2, muon_means, width, yerr=muon_stds, capsize=5,
               label='Muon-Exact', color=MUON_COLOR, edgecolor='black', alpha=0.85,
               align='center')
bars2 = ax.bar(x + width/2, sgd_means, width, yerr=sgd_stds, capsize=5,
               label='SGD', color=SGD_COLOR, edgecolor='black', alpha=0.85,
               align='center')

ax.set_xlabel('Initialization Type', fontsize=13)
ax.set_ylabel(r'$K\_epsilon$ (Iterations to Convergence)', fontsize=13)
ax.set_title('Convergence Speed by Initialization Type (E17)', fontsize=14)
ax.set_xticks(x)
ax.set_xticklabels([it.capitalize() for it in init_types])
ax.legend(fontsize=12)
ax.grid(True, alpha=0.3, axis='y')

# Add value labels
for i, (m, s) in enumerate(zip(muon_means, muon_stds)):
    ax.text(x[i] - width/2, m + s + 5, f'{m:.0f}', ha='center', va='bottom',
            color=MUON_COLOR, fontweight='bold')
for i, (m, s) in enumerate(zip(sgd_means, sgd_stds)):
    ax.text(x[i] + width/2, m + s + 5, f'{m:.0f}', ha='center', va='bottom',
            color=SGD_COLOR, fontweight='bold')

plt.tight_layout()
plt.savefig('E17_K_epsilon_by_init_type.png', dpi=150, bbox_inches='tight')
plt.show()
```

Visualization 2: Convergence Rate by Initialization Type

This plot shows the proportion of runs that successfully converged within the 2000-iteration budget for each initialization type and algorithm.

```
In [ ]: fig, ax = plt.subplots(figsize=(10, 6))

conv_data = df.groupby(['algo', 'init_type'])['I_conv'].mean().reset_index()
muon_conv = conv_data[conv_data['algo'] == 'Muon-Exact']
sgd_conv = conv_data[conv_data['algo'] == 'SGD']

x = np.arange(len(init_types))
width = 0.35
```

```

bars1 = ax.bar(x - width/2, muon_conv['I_conv'], width, label='Muon-Exact',
bars2 = ax.bar(x + width/2, sgd_conv['I_conv'], width, label='SGD', color=SGD_COLOR)

ax.set_xlabel('Initialization Type', fontsize=13)
ax.set_ylabel('Convergence Rate', fontsize=13)
ax.set_title('Convergence Success Rate by Initialization Type (E17)', fontsize=14)
ax.set_xticks(x)
ax.set_xticklabels([it.capitalize() for it in init_types])
ax.set_ylim(0, 1.05)
ax.legend(fontsize=12)
ax.grid(True, alpha=0.3, axis='y')

for bar in bars1:
    height = bar.get_height()
    ax.text(bar.get_x() + bar.get_width()/2., height + 0.01, f'{height:.2f}',
            ha='center', va='bottom', fontsize=10, fontweight='bold')
for bar in bars2:
    height = bar.get_height()
    ax.text(bar.get_x() + bar.get_width()/2., height + 0.01, f'{height:.2f}',
            ha='center', va='bottom', fontsize=10, fontweight='bold')

plt.tight_layout()
plt.savefig('E17_convergence_rate_by_init_type.png', dpi=150, bbox_inches='tight')
plt.show()

```

Visualization 3: Wall-Clock Time by Initialization Type

This plot compares the computational time required by each algorithm across initialization types. This is important because Muon's SVD computation has higher per-iteration cost than SGD.

```

In [ ]: fig, ax = plt.subplots(figsize=(10, 6))

muon_times = [df[(df['algo'] == 'Muon-Exact') & (df['init_type'] == it)]['time_s'].mean() for it in init_types]
muon_time_stds = [df[(df['algo'] == 'Muon-Exact') & (df['init_type'] == it)]['time_s'].std() for it in init_types]
sgd_times = [df[(df['algo'] == 'SGD') & (df['init_type'] == it)]['time_s'].mean() for it in init_types]
sgd_time_stds = [df[(df['algo'] == 'SGD') & (df['init_type'] == it)]['time_s'].std() for it in init_types]

bars1 = ax.bar(x - width/2, muon_times, width, yerr=muon_time_stds, capsize=5,
               label='Muon-Exact', color=MUON_COLOR, edgecolor='black', alpha=0.85)
bars2 = ax.bar(x + width/2, sgd_times, width, yerr=sgd_time_stds, capsize=5,
               label='SGD', color=SGD_COLOR, edgecolor='black', alpha=0.85)

ax.set_xlabel('Initialization Type', fontsize=13)
ax.set_ylabel('Wall-Clock Time (seconds)', fontsize=13)
ax.set_title('Computation Time by Initialization Type (E17)', fontsize=14, fontweight='bold')
ax.set_xticks(x)
ax.set_xticklabels([it.capitalize() for it in init_types])
ax.legend(fontsize=12)
ax.grid(True, alpha=0.3, axis='y')

for i, (m, s) in enumerate(zip(muon_times, muon_time_stds)):

```

```

    ax.text(x[i] - width/2, m + s + 0.5, f'{m:.1f}s', ha='center', va='bottom')
    for i, (m, s) in enumerate(zip(sgd_times, sgd_time_stds)):
        ax.text(x[i] + width/2, m + s + 0.5, f'{m:.1f}s', ha='center', va='bottom')

plt.tight_layout()
plt.savefig('E17_time_by_init_type.png', dpi=150, bbox_inches='tight')
plt.show()

```

Statistical Tests

Paired t-tests comparing Muon vs SGD for each initialization type, plus one-way ANOVA testing whether init_type has a significant effect within each algorithm. We also perform Tukey's HSD post-hoc tests to identify which init types differ significantly.

```

In [ ]: def cohens_d(x, y, paired=True):
    if paired:
        diff = x - y
        return diff.mean() / diff.std(ddof=1) if diff.std(ddof=1) > 0 else 0
    else:
        pooled_std = np.sqrt(((len(x)-1)*x.var(ddof=1) + (len(y)-1)*y.var(ddof=1)) / (len(x)+len(y)-2))
        return (x.mean() - y.mean()) / pooled_std if pooled_std > 0 else 0

# Paired t-tests: Muon vs SGD per init_type
print("=" * 95)
print(f"{'Init Type':>12} {'Muon mean':>11} {'SGD mean':>11} {'Diff':>9} {'Cohen's d':>11}")
print("=" * 95)

for it in df['init_type'].unique():
    muon_k = df[(df['algo'] == 'Muon-Exact') & (df['init_type'] == it)][['K_epsilon', 'K_epsilon_std']]
    sgd_k = df[(df['algo'] == 'SGD') & (df['init_type'] == it)][['K_epsilon', 'K_epsilon_std']]
    d = cohens_d(muon_k, sgd_k, paired=True)
    t_stat, p_val = stats.ttest_rel(muon_k, sgd_k)
    sig = "" if p_val < 0.001 else "" if p_val < 0.01 else "*" if p_val < 0.05 else ""
    print(f"{'it':>12} {'muon_k.mean()':>11.1f} {'sgd_k.mean()':>11.1f} {'muon_k.mean() - sgd_k.mean()':>11.1f} {'d':>9.3f} {'t_stat':>9.3f} {'p_val':>13.6f} {'sig':>5}")

# Post-hoc: compare init types within each algorithm using Tukey's HSD concept
print("\n=== Pairwise comparisons of init types (within each algorithm) ===")
from itertools import combinations

for algo in ['Muon-Exact', 'SGD']:
    print(f"\n{algo}:")
    sub = df[df['algo'] == algo]
    types = sub['init_type'].unique()
    for t1, t2 in combinations(types, 2):
        g1 = sub[sub['init_type'] == t1]['K_epsilon'].values
        g2 = sub[sub['init_type'] == t2]['K_epsilon'].values
        t_stat, p_val = stats.ttest_ind(g1, g2)
        sig = "" if p_val < 0.001 else "" if p_val < 0.01 else "*" if p_val < 0.05 else ""
        print(f"    {t1} vs {t2}: t={t_stat:+.3f}, p={p_val:.4f} {sig}")

```

Conclusions & Interpretation

Summary of Findings

1. **Best Initialization Type:** Based on mean K_ϵ , the best initialization type for each algorithm is identified in the summary table. Typically, orthogonal or spectral initialization provides a better starting point than random due to better conditioning.
2. **Muon vs SGD Consistency:** If Muon's K_ϵ values are more consistent across init types (lower standard deviation of means), this suggests spectral normalization provides stability regardless of initialization structure.
3. **Structured Initialization Benefit:** Orthogonal and spectral initializations may benefit both algorithms by providing well-conditioned starting points, but the relative improvement may differ:
 - Muon's spectral normalization may have **synergistic effects** with spectral initialization
 - SGD may benefit more from orthogonal initialization due to better gradient conditioning
4. **Statistical Significance:** The p-values from paired t-tests determine whether the Muon-SGD gap is significant for each init type. The ANOVA results test whether init type has a significant main effect.

Key Takeaway

The experiment reveals whether the choice of initialization matters more for Muon or SGD, and whether structured initializations (orthogonal/spectral) provide disproportionate benefits to either algorithm. If Muon shows similar advantages across all init types, this demonstrates that its spectral normalization is a robust optimization mechanism.

Limitations

- Only 8 seeds per configuration
- Fixed scale (0.01) for all structured initializations — varying scale could change results
- Only 3 init types tested — other strategies (Xavier, Kaiming, zero init) not explored